

Crypto Exchange v1.0

Índice

- 1. Arquitectura Base (Parte Obligatoria)
 - 1.1 Fuentes y Pares (Binance)
 - 1.2 Ingesta — Apache Kafka
 - 1.3 Almacenamiento — HDFS
 - 1.4 Procesamiento — Apache Spark Streaming
 - 1.5 Observabilidad — Prometheus y Grafana
- 2. Retos de Ampliación (Para Subir Nota)
 - 2.1 Enriquecimiento con CoinGecko (REST)
 - 2.2 Fear & Greed Index (Sentimiento)
- 3. Rúbrica de Evaluación
- 4. Entrega del Proyecto
- 5. Resultados de Aprendizaje y Criterios de Evaluación (RA / CE)

En este proyecto final vas a construir un sistema completo de ingesta, procesamiento y visualización de datos del mercado de criptomonedas. Para ello, utilizarás las APIs públicas de exchanges reales (como Binance y CoinGecko) para capturar la volatilidad del mercado en tiempo real.

El objetivo es no solo consumir los datos, sino orquestar todo el ecosistema Big Data **como un profesional del dato**: Kafka actuará como bus de eventos, Spark Streaming procesará flujos en vivo (calculando métricas como medias móviles o detectando *pump and dumps*), HDFS almacenará el histórico y Grafana mostrará tus resultados.



NOTA: ¿Por qué criptomonedas?

El volumen, velocidad temporal y variabilidad que generan mercados abiertos 24/7 como Binance los convierten en el ecosistema perfecto para llevar tus clústeres distribuidos al máximo rendimiento real.

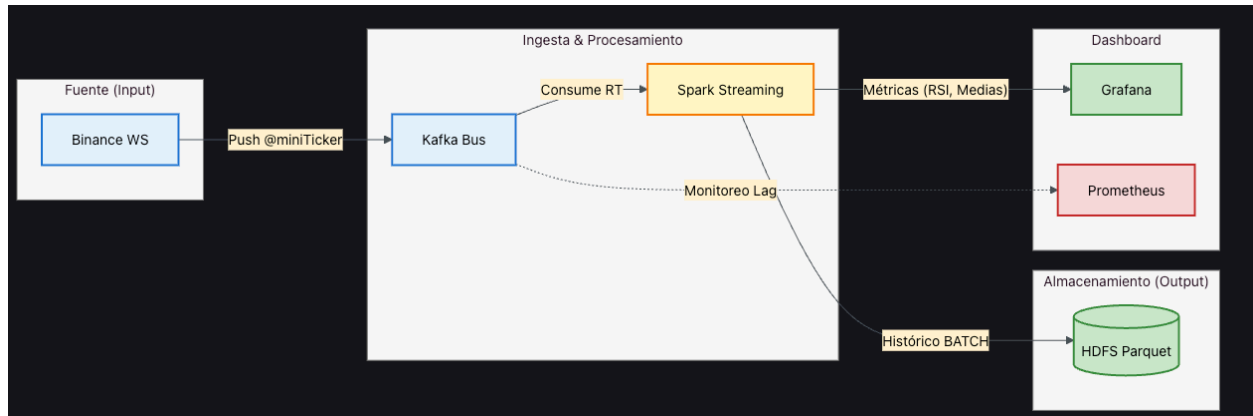


ATENCIÓN: Requisito técnico obligatorio

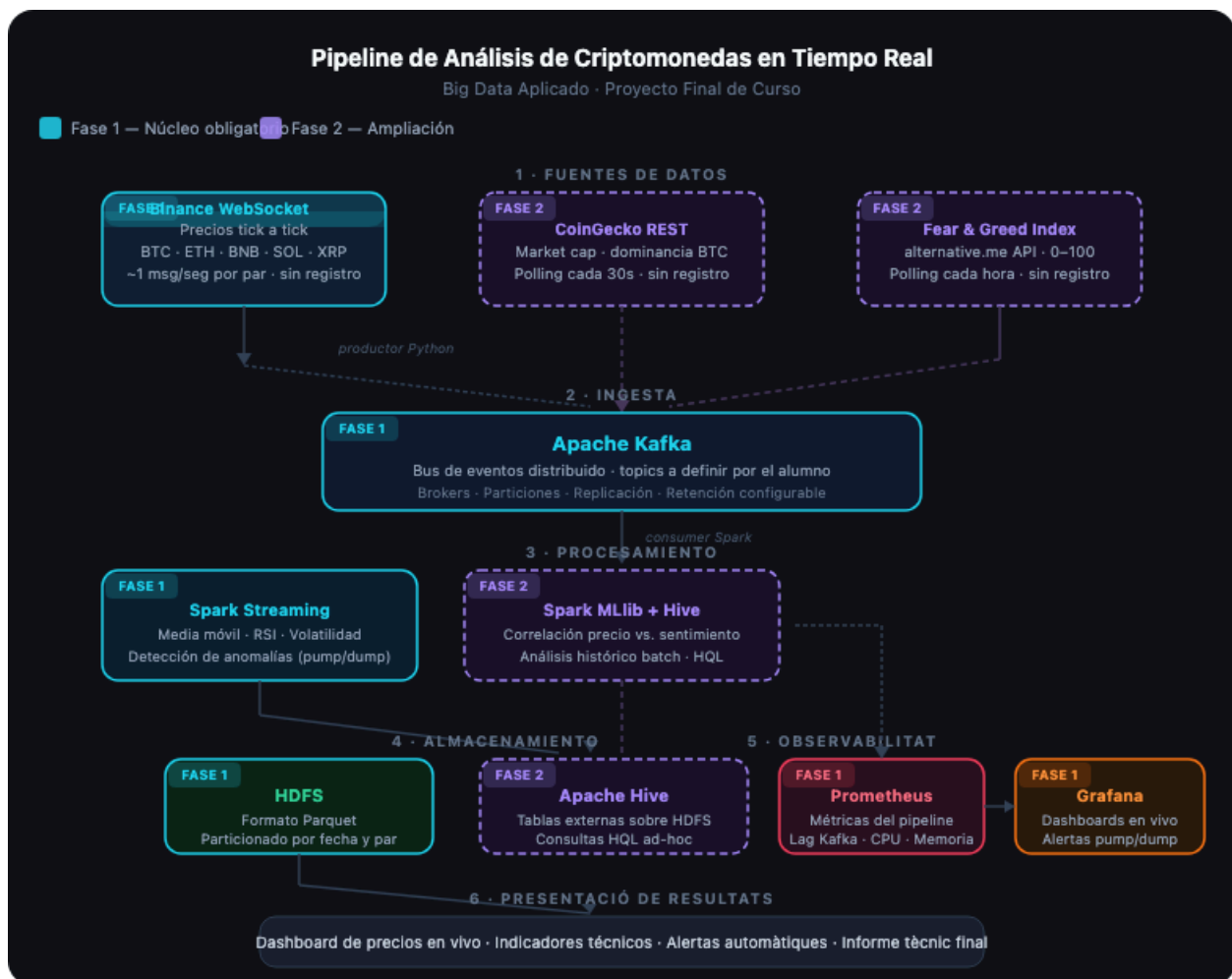
Todo el proyecto deberá estar montado sobre un **clúster Docker**, orquestado con **Docker Compose** (o similar). No se aceptarán entregas con servicios instalados directamente en la máquina. Cada componente del pipeline (Kafka, Spark, HDFS, Prometheus, Grafana...) debe arrancar como un contenedor independiente definido en el `docker-compose.yml`.

1. Arquitectura Base (Parte Obligatoria)

La arquitectura principal que **debes implementar para aprobar** se basa en un flujo unidireccional de datos en tiempo real.



Arquitectura base + ampliación



1.1 Fuentes y Pares (Binance)

No es necesario registrarse en Binance. La parte de Binance que sí requiere API es la parte del exchange (comprar, vender, etc.), no la parte de obtener datos.

Deberás monitorizar **2 pares de tokens**.

1. Uno obligatorio: BTC/USDT
2. Otro de libre elección (ej. ETH/USDT, SOL/USDT).

Conéctate vía [Binance WebSocket](#). **No necesitas registrarte ni usar API Keys** para leer los streams públicos:

Stream	Qué envía	Frecuencia
@miniTicker	Precio actual, volumen 24h, % cambio	Cada ~1 segundo
@kline_1m	Vela OHLCV de 1 minuto	Cada segundo (vela abierta)



IMPORTANTE: Cuidado con eventos hiperactivos

Para este proyecto no utilizaremos @trade ni @depth (también existentes en la API), ya que emiten cada operación individual e inundarán nuestro bus de Kafka inmediatamente en un entorno local.

Puedes comprobar que el *stream* funciona de manera muy sencilla por consola antes de codificar en Python:

```
# Con wscat (npm install -g wscat)
wscat -c "wss://stream.binance.com:9443/ws/btcusdt@miniTicker"
```



Entregable mínimo:

- Conexión WebSocket establecida y estable
- Suscripción a @miniTicker para BTC/USDT y el par de libre elección
- Suscripción a @kline_1m para el cálculo posterior de indicadores
- Productor Python publicando mensajes en Kafka sin errores
- Captura de pantalla o log que evidencie el stream recibido

1.2 Ingesta - Apache Kafka

El primer servicio será instanciar Kafka actuando como **bus de eventos distribuido**. Tus productores en Python publicarán mensajes desde la API de Binance al *topic*.

Una decisión **muy importante de diseño que deberás justificar** en tu entrega: ¿usarás un *topic* general? ¿uno por cada par de trading? ¿topics separados por tipo de dato? Cada opción tiene implicaciones en el rendimiento, en la escalabilidad y en la complejidad de gestión.

Para el proyecto desplegarás Kafka mediante **Docker Compose** con configuración mínima de 1 broker, suficiente para demostrar el funcionamiento completo del pipeline en entorno local.

✓ Entregable mínimo:

- Clúster Kafka operativo con Docker Compose
- Diseño de topics implementado y **justificado por escrito en la memoria**
- Mensajes de Binance llegando correctamente al topic
- Consumidor Spark conectado y leyendo el flujo en tiempo real

1.3 Almacenamiento - HDFS

HDFS es la capa de **persistencia distribuida** del pipeline. Todo lo que Spark procesa en tiempo real tiene una vida efímera en memoria — HDFS es donde los datos sobreviven, se acumulan y quedan disponibles para análisis posteriores.

En este proyecto HDFS cumple dos funciones:

- **Histórico de precios procesados:** cada micro-batch que Spark escribe queda almacenado de forma permanente, construyendo un histórico que crece con el tiempo.
- **Base para consultas batch con Hive:** sobre esos ficheros en HDFS se definen tablas externas en Hive (módulo 1), permitiendo lanzar consultas HQL sobre semanas o meses de datos.

Formato de almacenamiento

Los datos se escriben en formato **Parquet**, no en CSV ni JSON. Las razones son:

Característica	CSV / JSON	Parquet
Compresión	Sin comprimir o básica	Columnar, muy eficiente
Velocidad de lectura	Lee todas las columnas	Lee solo las columnas necesarias
Esquema	Implícito, propenso a errores	Embebido en el fichero
Compatibilidad Spark/Hive	Sí, pero lento	Nativo y optimizado

Para datos de series temporales con muchas columnas repetitivas (precio, volumen, par, timestamp...) Parquet puede reducir el tamaño hasta un 80% respecto a JSON.

Estructura de directorios y particionado

Spark puede escribir los ficheros organizados en carpetas automáticamente según los valores de una columna. A esta organización se le llama **particionado** y es una decisión de diseño clave:

 Para trabajar con diferentes días puedes modificar el timestamp del mensaje que llega a Kafka.

```
/data/  
  cripto/  
    par=BTCUSDT/  
      fecha=2025-01-15/  
        part-00000.parquet  
        part-00001.parquet  
    par=ETHUSDT/  
      fecha=2025-01-15/  
        part-00000.parquet
```

Con esta estructura, una consulta que filtre por par=BTCUSDT solo lee esa carpeta, sin tocar el resto del histórico.

NOTA: ¿Qué debes justificar en la memoria?

Al igual que con los topics de Kafka, no hay una única respuesta correcta. Para cada decisión deberás argumentar:

1. **¿Por qué Parquet y no otro formato?** Compara al menos con CSV. ¿Qué pierdes? ¿Qué ganas?
2. **¿Cómo has particionado?** ¿Por fecha, por par, por ambos? ¿Qué ocurre si solo particionas por fecha y tienes 10 pares? ¿Y si solo por par y tienes 2 años de datos?
3. **¿Qué factor de replicación has configurado?** HDFS replica los bloques por defecto 3 veces. En un clúster Docker de desarrollo con 1 nodo, ¿tiene sentido mantenerlo? ¿Qué implica reducirlo a 1?
4. **¿Cuánto tiempo conservas los datos?** ¿Borramos datos antiguos o acumulamos indefinidamente? ¿Qué impacto tiene en el tamaño del clúster?

Entregable mínimo:

- Spark escribiendo en HDFS en formato Parquet sin errores
- Estructura de directorios particionada y justificada en la memoria
- Factor de replicación configurado y razonado
- Ficheros legibles desde la shell HDFS (`hdfs dfs -ls`, `hdfs dfs -du`)
- Al menos una consulta HQL sobre los datos históricos almacenados

1.4 Procesamiento - Apache Spark Streaming

Con los datos fluyendo por Kafka, Spark Streaming actúa como el **cerebro analítico del pipeline**. Mediante **Structured Streaming** (la API moderna de Spark), te suscribes al topic de Kafka como si fuera una tabla en continua actualización y aplicas transformaciones en tiempo real sobre cada micro-batch recibido.

El procesamiento tiene dos salidas diferenciadas según el diagrama de arquitectura:

- **Salida en tiempo real** → **Grafana**: métricas calculadas al vuelo que se visualizan en el dashboard en vivo. El mecanismo para exponer estas métricas hacia Grafana se detalla en la [sección 1.5](#).
- **Salida histórica** → **HDFS**: los datos procesados se escriben en formato **Parquet**, particionados por fecha y par de trading, tal como se describe en la [sección 1.3](#).

Algunos cálculos que puedes implementar sobre la ventana de datos recibida:

Indicador	Qué detecta	Dificultad
Media móvil (SMA/EMA)	Tendencia general del precio	Básica
Variación porcentual	Cambio brusco en el último tick	Básica
RSI (Relative Strength Index)	Sobrecompra / sobreventa	Media
Bollinger Bands	Volatilidad anómala	Media
Detección pump & dump	Subida/caída > X% en < Y segundos	Avanzada

NOTA: Ventanas temporales

Spark Streaming no procesa cada mensaje de forma aislada, sino que agrupa los eventos en **ventanas de tiempo** para calcular estadísticas sobre un periodo.

Una ventana se define con dos parámetros: - **Duración** (window duration): cuánto tiempo de datos incluye la ventana. Ejemplo: los últimos 5 minutos. - **Deslizamiento** (slide duration): cada cuánto se recalcula. Ejemplo: cada 30 segundos.

Con esos dos valores, Spark calcula el indicador sobre los datos de los últimos 5 minutos, y lo actualiza cada 30 segundos — sin esperar a que la ventana se cierre del todo.

```
|--- 5 min ---|
[t-5min ----- t-4:30 ----- ... ----- t] → cálculo
      |--- 5 min ---|
      [t-4:30 ----- ... ----- t+0:30] → siguiente cálculo
```

¿Qué debes justificar en la memoria?

No existe una configuración correcta universal, depende del uso. Para cada indicador que implementes deberás razonar:

1. **¿Por qué esa duración?** Una ventana de 1 minuto detecta movimientos bruscos pero es ruidosa. Una de 1 hora suaviza demasiado y pierde reactividad.
2. **¿Por qué ese deslizamiento?** Si el deslizamiento es igual a la duración, las ventanas no se solapan (tumbling window). Si es menor, se solapan (sliding window) y hay más recálculos.
3. **¿Qué impacto tiene en el rendimiento?** Ventanas largas con deslizamiento corto generan más carga en Spark. ¿Tu clúster Docker lo aguanta?
4. **¿Tiene sentido para el activo que monitorizas?** BTC puede moverse un 3% en 2 minutos. Un par menos líquido puede necesitar una ventana mayor para detectar la misma señal.

✓ Entregable mínimo:

- Spark Streaming leyendo correctamente del topic de Kafka
- Al menos **dos indicadores técnicos** calculados sobre el flujo (ej. media móvil + variación %)
- Datos procesados escritos en HDFS en formato Parquet
- Particionado por fecha y par de trading justificado en la memoria
- Métricas visibles en tiempo real en Grafana (ver sección 1.5)

1.5 Observabilidad - Prometheus y Grafana

Con el pipeline funcionando, necesitas saber en todo momento si está funcionando **bien**. ¿Está Kafka recibiendo mensajes? ¿Spark tiene lag acumulado? ¿Se está quedando sin memoria algún contenedor? Sin monitorización, un fallo silencioso puede llevar horas corrompiendo datos sin que te enteres.

Prometheus y Grafana resuelven esto formando un stack de observabilidad completo:

- **Prometheus** va preguntando periódicamente a cada servicio por sus métricas (modelo pull) y las almacena como series temporales.
- **Grafana** lee esas series temporales y las representa en dashboards visuales en tiempo real.

En este proyecto la monitorización tiene **dos capas diferenciadas** que debes implementar por separado:

Capa 1 — Infraestructura (el clúster Docker)

Métricas de los contenedores y el sistema operativo: CPU, memoria, red, disco. Se obtienen con dos exporters estándar que no requieren configuración adicional:

Exporter	Qué monitoriza	Puerto
Node Exporter	CPU, RAM, disco y red del host	9100
cAdvisor	Recursos consumidos por cada contenedor Docker	8080

Capa 2 — Negocio (el pipeline de datos)

Métricas propias del proyecto: precio actual de BTC, número de mensajes procesados por Spark, lag del consumer de Kafka, anomalías detectadas. Estas métricas **no existen por defecto** — las genera el propio código Python o Spark usando la librería `prometheus_client`.

```
from prometheus_client import Gauge, Counter, start_http_server

precio_btc = Gauge('crypto_precio_actual', 'Precio actual', ['par'])
mensajes_procesados = Counter('pipeline_mensajes_total', 'Total mensajes procesados')

# Al recibir cada mensaje de Kafka:
precio_btc.labels(par='BTCUSD').set(precio)
mensajes_procesados.inc()
```

Prometheus recoge estas métricas automáticamente desde `http://tu-contenedor:8000/metrics`.

NOTA: ¿Qué debes justificar en la memoria?

1. **¿Qué métricas de negocio has expuesto?** No basta con instalar Node Exporter. Debes definir al menos 3 métricas propias del pipeline y razonar por qué son relevantes para detectar problemas.
2. **¿Qué alertas has configurado?** Una alerta bien definida tiene un umbral, una duración y una severidad. Por ejemplo: *“Si el lag del consumer de Kafka supera 1000 mensajes durante más de 2 minutos, es crítico”*. ¿Por qué esos valores y no otros?
3. **¿Cómo organizas los dashboards?** ¿Un dashboard único para todo o separados por capa (infraestructura / negocio)? ¿Por qué?

Entregable mínimo:

- Node Exporter y cAdvisor desplegados y scrapeados por Prometheus
- Al menos **3 métricas de negocio** propias expuestas desde el pipeline
- Dashboard Grafana con métricas de infraestructura (CPU, memoria por contenedor)
- Dashboard Grafana con métricas de negocio (precio en vivo, mensajes procesados)
- Al menos **1 alerta** configurada en Prometheus con umbral justificado en la memoria

2. Ampliación (Para Subir Nota)

Una vez que tengas el pipeline de la Fase 1 estable y funcionando, puedes incorporar fuentes de datos adicionales para enriquecer el análisis. Cada uno de estos retos es independiente — puedes implementar uno, los dos, o ninguno — pero cada uno bien justificado suma significativamente a la nota.

2.1 Enriquecimiento con CoinGecko (REST)

Binance te da el precio tick a tick de tus pares de trading, pero no el contexto global del mercado. ¿Está BTC subiendo porque hay interés real o porque domina el 60% del mercado y arrastra todo? Para responder eso necesitas datos agregados que Binance no proporciona.

CoinGecko ofrece una API REST pública y gratuita (sin registro) con métricas de mercado globales: capitalización total, dominancia de BTC, volumen agregado de todos los exchanges, ranking de criptomonedas...

A diferencia de Binance, aquí no hay WebSocket — los datos no cambian segundo a segundo, así que usarás **polling periódico**: una petición REST cada 30-60 segundos que publica el resultado en Kafka como un productor independiente.

Precio y market cap de BTC y ETH en una sola petición

curl

"https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum&vs_currencies=usd&include_market_cap=true&include_24hr_vol=true"

```
{
  "bitcoin": { "usd": 67420, "usd_market_cap": 1327000000000, "usd_24h_vol": 28000000000 },
  "ethereum": { "usd": 3540, "usd_market_cap": 425000000000, "usd_24h_vol": 14000000000 }
}
```

Con estos datos puedes hacer un **join** en Spark o directamente en HDFS entre el precio de Binance y el contexto de CoinGecko, respondiendo preguntas como: *¿El precio de BTC en Binance se desvía del precio global agregado por CoinGecko? ¿Cuándo y cuánto?*

 **NOTA: ¿Qué debes justificar en la memoria?**

1. **¿Cada cuánto haces polling?** Cada 30 segundos, cada minuto, cada 5 minutos... ¿Qué granularidad tiene sentido para datos que cambian lentamente? ¿Qué pasa si pones un intervalo demasiado corto?
2. **¿Dónde haces el join?** ¿En Spark Streaming cruzando ambos flujos en tiempo real, o en HDFS cruzando los históricos en batch con Hive? ¿Qué ventajas e inconvenientes tiene cada opción?
3. **¿Qué pregunta de negocio responde este enriquecimiento?** Define una hipótesis concreta y demuestra con datos si se cumple o no.

★ Entregable ampliación:

- Productor REST con polling periódico a CoinGecko implementado y desplegado en Docker
- Datos de market cap y dominancia BTC disponibles en el pipeline - Join implementado (en Spark o Hive) justificado en la memoria
- Al menos una pregunta de negocio respondida con los datos cruzados

2.2 Fear & Greed Index (Sentimiento)

El análisis técnico (precios, volúmenes, indicadores) te dice *qué* está pasando en el mercado. El análisis de sentimiento te dice *por qué*: ¿están los inversores con miedo o con euforia?

En lugar de implementar modelos complejos de PLN sobre textos de Reddit o Twitter, vamos a incorporar el **Fear & Greed Index** de [alternative.me](https://alternative.me/fng/): un índice numérico entre 0 y 100 que agrega señales de volatilidad, volumen, redes sociales y dominancia para resumir el sentimiento del mercado cripto en un único valor.

Valor	Clasificación
0 – 24	Extreme Fear (miedo extremo)
25 – 49	Fear (miedo)
50 – 74	Greed (codicia)
75 – 100	Extreme Greed (codicia extrema)

Índice actual + histórico de los últimos 10 días

```
curl "https://api.alternative.me/fng/?limit=10"
```

```
{
  "data": [
    { "value": "72", "value_classification": "Greed", "timestamp": "1711756800" },
    { "value": "65", "value_classification": "Greed", "timestamp": "1711670400" }
  ]
}
```

Con un polling horario acumulas el histórico en una tabla Hive y puedes cruzarlo con los datos de precio y volumen almacenados en HDFS. La pregunta de negocio que debes responder es:

¿Caen los volúmenes de trading cuando el índice refleja miedo extremo? ¿Sube el precio cuando hay codicia extrema?

 NOTA: ¿Qué debes justificar en la memoria?

1. **¿Con qué frecuencia haces polling?** El índice se actualiza una vez al día, así que hacer polling cada minuto no tiene sentido. ¿Cada cuánto es suficiente?
2. **¿Cómo modelas la tabla Hive?** ¿Qué columnas incluyes? ¿Guardas solo el valor numérico o también la clasificación textual?
3. **¿Qué conclusión extraes del cruce de datos?** No basta con hacer la query — debes interpretar el resultado e incluirlo en la memoria con una conclusión razonada.

★ **Entregable ampliación:**

- Polling a `api.alternative.me/fng/` implementado y desplegado en Docker
- Histórico del índice almacenado en tabla Hive
- Query HQL que cruce el índice con los datos de precio/volumen de HDFS
- Conclusión razonada sobre la hipótesis planteada, visualizada en Grafana o en la memoria

3. Rúbrica de Evaluación

Parte 1 — Arquitectura Base (Obligatoria)

Criterio	Nivel 1 (Insuficiente)	Nivel 2 (Aprobado)	Nivel 3 (Notable)	Nivel 4 (Excelente)
Fuente de datos (Binance)	No hay conexión WebSocket o se desconecta constantemente.	Conexión estable a @miniTicker para al menos un par.	Suscripción a @miniTicker y @kline_1m para los dos pares, productor Python estable.	Reconexión automática ante caídas, logs de errores gestionados, latencia medida y documentada.
Ingesta (Kafka)	No existe clúster Kafka o los mensajes no llegan al topic.	Mensajes publicados y consumidos en al menos un topic.	Diseño de topics justificado en la memoria, particiones configuradas correctamente.	Lag del consumer monitorizado por Prometheus, retención configurada y razonada en la memoria.
Almacenamiento (HDFS)	Se escriben archivos .txt o sin clúster Hadoop distribuido.	Datos escritos correctamente en formato Parquet sobre HDFS.	Histórico particionado por par y/o fecha con queries HQL funcionales sobre tablas Hive.	Factor de replicación justificado, estructura de directorios razonada, consultas analíticas complejas en Hive.

Procesamiento (Spark Streaming)	Los esquemas originan fallos de inferencia o no hay transformaciones.	Procesa el flujo y calcula al menos un indicador técnico básico.	Implementa ventanas temporales con duración y deslizamiento justificados en la memoria.	Dos o más indicadores con ventanas diferentes, detección de pump & dump funcionando y documentada.
Observabilidad (Prometheus + Grafana)	No hay monitorización o solo el dashboard por defecto de Grafana.	Node Exporter y cAdvisor desplegados, precio actual visible en Grafana.	Al menos 3 métricas de negocio propias expuestas con prometheus_client, alerta configurada.	Dos dashboards diferenciados (infraestructura / negocio), alertas con umbral y severidad justificados, PromQL razonado en la memoria.
Documentación y memoria	Sin memoria o sin justificaciones de diseño.	Memoria con descripción del proyecto y capturas de pantalla.	Todas las decisiones de diseño justificadas (topics, particionado, ventanas, métricas).	Diagrama de arquitectura propio, sección de limitaciones y mejoras futuras razonada.

Parte 2 — Ampliación (Opcional, para subir nota)

Esta rúbrica solo se aplica si has implementado alguno de los retos de la sección 2. Cada reto se evalúa de forma independiente.

Criterio	Nivel 1 (Incompleto)	Nivel 2 (Funcional)	Nivel 3 (Bien justificado)	Nivel 4 (Sobresaliente)
Integración CoinGecko	Productor REST implementado pero sin datos en el pipeline.	Datos de market cap y dominancia disponibles en Kafka o HDFS.	Frecuencia de polling justificada, join con datos de Binance implementado.	Pregunta de negocio planteada, respondida con datos y concluida en la memoria con visualización en Grafana.
Fear & Greed Index	Polling implementado pero sin almacenamiento o histórico.	Histórico del índice cargado en tabla Hive correctamente.	Query HQL que cruza el índice con precio y volumen de HDFS.	Hipótesis planteada, query ejecutada, conclusión razonada e interpretada en la memoria con visualización.
Calidad del análisis cruzado	Los datos de ambas fuentes no se relacionan entre sí.	Join implementado técnicamente, aunque sea en batch con Hive.	Join justificado (¿en Spark Streaming o en Hive? ¿Por qué?), resultado coherente.	Análisis comparativo entre las dos fuentes con conclusión de negocio clara y bien argumentada.

4. Entrega del Proyecto

El proyecto se entregará en un **repositorio público de GitHub** de cada alumno.

Entregable 1 — Repositorio GitHub

El repositorio debe contener como mínimo:

- `docker-compose.yml` en la raíz — el proyecto debe arrancar con `docker compose up` sin pasos adicionales
- Carpetas separadas por servicio (`/producer`, `/spark`, `/grafana...`)
- `README.md` con instrucciones claras para levantar el proyecto
- `memoria.pdf` con la documentación técnica (ver Entregable 2)

Entregable 2 — Memoria técnica (PDF)

La memoria debe incluir:

- Resumen del proyecto realizado
- Justificación de todas las decisiones de diseño: topics Kafka, particionado HDFS, ventanas Spark, métricas Prometheus
- Diagrama de arquitectura propio — cómo quedó realmente implementado, no el del enunciado
- Sección de limitaciones y posibles mejoras

Entregable 3 — Presentación y defensa

La defensa del proyecto se estructura en **20 minutos**:

Bloque	Duración	Descripción
 Presentación PowerPoint	10 min	Arquitectura, decisiones de diseño y resultados obtenidos
 Demo en vivo	5 min	El pipeline arrancado y funcionando, con datos reales llegando a Grafana en tiempo real
 Preguntas	5 min	Preguntas del profesor sobre cualquier aspecto del proyecto

ATENCIÓN: Sobre la demo

La demo debe realizarse en vivo durante la presentación. No se aceptará un vídeo grabado como sustituto. El pipeline debe estar operativo y mostrando datos reales en el momento de la defensa.

5. Resultados de Aprendizaje y Criterios de Evaluación (RA / CE)

Este proyecto cubre los siguientes Resultados de Aprendizaje y Criterios de Evaluación del módulo:

RA 0 — Fa servir el terminal de Linux

CE	Descripció	On s'aplica al projecte
a	S'han utilitzat comandaments	Gestió de contenidors Docker, HDFS shell, logs de serveis
b	S'han realitzat tasques administratives des del terminal	Arrencada i parada del clúster, comprovació de ports i processos
c	S'han programat scripts	Scripts Bash d'automatització per arrencar el pipeline complet

RA 1 — Gestiona solucions d'emmagatzematge i eines associades al centre de dades

CE	Descripció	On s'aplica al projecte
a	S'ha caracteritzat el procés de disseny i construcció de solucions	Disseny i justificació de l'arquitectura completa del pipeline
b	S'han determinat els procediments i mecanismes per a la ingestió de dades	Connexió WebSocket a Binance, producció de missatges a Kafka
c	S'ha determinat el format de dades adequat per a l'emmagatzematge	Elecció i justificació del format Parquet sobre HDFS
d	S'han processat les dades emmagatzemades	Spark Streaming: indicadors tècnics, detecció d'anomalies
e	S'han presentat els resultats en una forma fàcil d'interpretar	Dashboards Grafana amb preus en viu i alertes

RA 2 — Gestiona sistemes d'emmagatzematge distribuïts per processar grans quantitats de dades

CE	Descripció	On s'aplica al projecte
a	S'ha determinat la importància dels sistemes d'emmagatzematge distribuïts	HDFS com a capa de persistència del pipeline
b	S'ha comprovat el poder de processament del model de computació distribuïda	Spark Structured Streaming sobre el clúster
c	S'ha provat la tolerància a fallades dels sistemes	Factor de replicació HDFS, particions Kafka
d	S'ha determinat que es poden emmagatzemar tantes dades com es desitgi	Escalabilitat del clúster HDFS per acumular històric
e	S'ha visualitzat que el sistema pot créixer fàcilment afegint mòduls	Arquitectura Docker Compose extensible (afegir brokers, workers)

RA 3 — Genera mecanismes d'integritat de les dades en sistemes de fitxers distribuïts

CE	Descripció	On s'aplica al projecte
a	S'ha valorat la importància de la qualitat de les dades	Validació dels missatges rebuts de Binance abans de publicar a Kafka
b	S'ha valorat que a major volum correspon un major perill d'integritat	Anàlisi del risc d'errors en el pipeline amb dades de mercat en temps real
c	S'ha reconegut que HDFS implementa sumes de verificació	Configuració i comprovació dels checksums de blocs HDFS
d	S'ha reconegut el paper del servidor en els processos de verificació	Rol del NameNode en la gestió de metadades i integritat

RA 4 — Realitza el seguiment del monitoratge d'un sistema

CE	Descripció	On s'aplica al projecte
a	S'han aplicat eines de monitoratge eficient dels recursos	Node Exporter + cAdvisor per a infraestructura Docker
b	S'han recollit mètriques, processament i visualització	Mètriques pròpies del pipeline exposades amb prometheus_client
c	S'han generat alertes per a detectar un risc o mal funcionament	Alertes Prometheus: lag Kafka, pump/dump detectat, contenidor caigut
d	S'ha comprovat que les eines ofereixen un rendiment elevat	Consultes PromQL en temps real sobre el dashboard Grafana
e	S'ha comprovat la fiabilitat de les dades segons respostes	Verificació de la coherència entre mètriques de Prometheus i dades reals
f	S'ha analitzat l'estabilitat de serveis	Monitorització contínua de tots els contenidors del clúster

RA 5 — Valida les tècniques de Big Data per transformar dades en informació significativa

CE	Descripció	On s'aplica al projecte
a	S'han seleccionat grans quantitats de dades estructurades i no estructurades	Dades de mercat de Binance (streaming) i indicadors calculats
b	S'ha realitzat la neteja i transformació de dades	Transformacions Spark: normalització, càlcul d'indicadors, filtratge
c	S'ha comprovat que el Big Data multiplica la rellevància del BI	Dashboard de negoci: senyals accionables a partir de dades brutes
e	S'ha avaluat i interpretat la informació extreta	Anàlisi dels indicadors tècnics i la seva influència en la presa de decisions
f	S'ha simulat la implantació d'un model d'Intel·ligència de negocis	Pipeline complet des de la font de dades fins a la visualització executiva

RA 6 — Desenvolupa un projecte integrador combinant IA i anàlisi de dades massives

CE	Descripció	On s'aplica al projecte
a	Defineix el problema i els objectius del projecte	Memòria: contextualització, motivació i viabilitat tècnica
b	Selecciona i justifica les fonts de dades	Justificació de Binance WebSocket vs. alternatives
e	Desenvolupa sistemes de Big Data per a ingesta, emmagatzematge i processament	Pipeline complet: Kafka → Spark → HDFS
f	Integra els components en una solució funcional i documentada	<code>docker-compose.yml</code> + memòria tècnica del projecte
g	Analitza i comunica els resultats obtinguts	Dashboards Grafana + informe tècnic final
h	Avalua críticament les limitacions i proposa millores futures	Secció de limitacions i línies d'ampliació a la memòria
i	Treballa de forma col·laborativa i ètica	Repositori GitHub públic, llicència, protecció de dades de mercat